Dokumentacja Projektu Aplikacji Internetowej

Encyklopedia Roślin i Wirtualny Asystent Ogrodnika (Ekosystem Warstwowy)

Przedmiot: Programowanie zaawansowane 2

Technologia: ASP.NET Core 9.0 MVC & REST API

Baza danych: Entity Framework Core / SQLite

Autorzy: Zespół Projektowy (Max 2 osoby)

Status: Ukończony (Laboratorium 11/12)

Rok akademicki: 2025/2026

1. Opis i przeznaczenie aplikacji

Projekt realizuje zaawansowany system wsparcia projektowania oraz zarządzania ogrodami permakulturowymi i ogrodami leśnymi (tzw. *food forests*). Głównym celem aplikacji jest dostarczenie użytkownikom narzędzia do optymalizacji nasadzeń pod kątem dwóch fundamentalnych kryteriów ekologicznych: **warstwowości roślinności** (piętrowego układu lasu) oraz **zależności allelopatycznych** (wzajemnego biochemicznego wpływu roślin na siebie).

System nie tylko działa jako klasyczna, encyklopedyczna baza danych, ale przede wszystkim implementuje dedykowaną logikę biznesową analizującą stan wirtualnego ogrodu każdego zalogowanego użytkownika. Aplikacja ułatwia planowanie zrównoważonych upraw poprzez automatyczne generowanie alertów o konfliktach sąsiedztwa i dostarczanie spersonalizowanych harmonogramów pielęgnacyjnych.

2. Struktura i model relacyjny bazy danych

Zgodnie z wymogami formalnymi instrukcji laboratoryjnej, aplikacja korzysta z relacyjnej bazy danych SQLite, a komunikacja z nią odbywa się w całości za pośrednictwem warstwy mapowania obiektowo-relacyjnego **Entity Framework Core** (brak natywnych kwerend SQL w kodzie). System operuje na **6 powiązanych tabelach** (limit min. 4 tabel dla projektu głównego został w pełni spełniony, z wyłączeniem tabeli logowania).

> **Zasada integralności danych:** Wszystkie powiązania zostały zabezpieczone przed anomaliami. W tabeli powiązań własnych (Allelopatia) zastosowano mechanizm `DeleteBehavior.Restrict`, chroniący bazę przed pętlami kaskadowego usuwania.

Opis poszczególnych encji (modeli):

Nazwa modelu (Tabeli)	Typ relacji i powiązania	Przeznaczenie i kluczowe pola
Warstwa	Relacja 1:N z tabelą `Rosliny`	Definiuje piętra roślinności w ekosystemie (np. Drzewa wysokie, Krzewy, Runo, Pnącza). Pola: Id, Nazwa, Opis.
Roslina	Klucz obcy do `Warstwy`, relacje 1:1 z `Wymagania`, 1:N z `Zaleznosci` oraz `Ogrody`	Główna encja encyklopedyczna. Przechowuje polską i łacińską nazwę oraz ogólne parametry botaniczne.
Wymaganie	Relacja 1:1 z tabelą `Rosliny` (ścisłe powiązanie)	Specyfikacja siedliskowa danego gatunku. Pola: Naslonecznienie, RodzajGleby, ZapotrzebowanieNaWode, Mrozoodpornosc.
Zaleznosc	Powiązanie samoreferencyjne (Self-referencing) 1:N z `Rosliny`	Modeluje allelopatię. Łączy dwie rośliny (Główną i Towarzyszącą) i określa typ wpływu (Pozytywna/Negatywna) wraz z opisem.
Uzytkownik	Relacja 1:N z tabelą `OgrodyUzytkownikow`	Przechowuje dane kont (Nazwa, Rola w systemie, bezpieczny skrót HasloHash oraz unikalny ApiToken autoryzacyjny).
OgrodUzytkownika	Tabela łącząca (Klucze obce do `Uzytkownicy` oraz `Rosliny`)	Zindywidualizowany rejestr nasadzeń konkretnego ogrodnika. Przechowuje ilość sztuk, datę posadzenia oraz notatki.

3. Implementacja zaawansowanej logiki (Dashboard)

System realizuje kryterium "robienia czegoś więcej" poprzez transformację strony głównej w inteligentny, dynamiczny panel analityczny (Dashboard). Po zalogowaniu system pobiera kontekst zalogowanej osoby i generuje trzy niezależne zestawienia:

1. **Audyt sąsiedztwa (Allelopatia w praktyce):** Algorytm analizuje zestaw unikalnych identyfikatorów roślin posadzonych przez danego użytkownika i krzyżowo sprawdza je z tabelą `Zaleznosci` dla typu "Negatywna". W przypadku wykrycia konfliktu (np. jednoczesne posiadanie pomidora i ziemniaka), system generuje wyraźny alert ostrzegawczy z opisem biologicznym zagrożenia.
2. **Statystyki warstwowości:** Aplikacja agreguje sumaryczną ilość sztuk wszystkich nasadzeń użytkownika i oblicza procentowy udział poszczególnych pięter ekosystemu. Wynik prezentowany jest za pomocą skumulowanego paska postępu (Stacked Progress Bar) w technologii Bootstrap, dając wizualny pogląd na to, czy ogród jest poprawnie zbalansowany ekologicznie.

3. **Wirtualny kalendarz pielęgnacji:** Rośliny z ogrodu użytkownika są automatycznie grupowane według kryterium zapotrzebowania na wodę pobieranego bezpośrednio z tabeli `Wymagania`, tworząc przejrzysty, kafelkowy podział zadań dla ułatwienia organizacji nawadniania.

4. Bezpieczeństwo, sesje i autoryzacja widoków

Aplikacja została w pełni zabezpieczona przed nieautoryzowanym dostępem na poziomie serwera przy użyciu mechanizmu filtrów akcji (Action Filters). Kontrola opiera się na stanowym mechanizmie `HttpContext.Session` z flagami bezpieczeństwa `HttpOnly` oraz `IsEssential`.

- **Filtr `[Zalogowany]`:** Blokuje bezpośredni dostęp do widoków zarządzania danymi (rośliny, wymagania, ogród) dla osób niezalogowanych, automatycznie przekierowując je do formularza autentykacji.
- **Filtr `[AdminTylko]`:** Nakłada restrykcje na kontroler `UzytkownikController`. Wyłącznie użytkownicy z rolą "Admin" mogą wyświetlać listę kont oraz rejestrować nowych użytkowników.
- **Bezpieczeństwo haseł:** Hasła użytkowników wprowadzane przez Administratora w formularzu Web UI są natychmiast przechwytywane przez kontroler i nieodwracalnie hashowane kryptograficznie algorytmem **SHA256** przed zapisem do bazy danych.

5. Dokumentacja interfejsu REST API

Aplikacja udostępnia pełnoprawny, bezstanowy interfejs programistyczny REST API dedykowany do integracji z zewnętrznymi systemami i automatyzacji zadań CRUD dla tabeli `Warstwa`.

Autentykacja i zabezpieczenie przed Over-Postingiem

Zgodnie z wymaganiami projektowymi, autoryzacja żądań REST odbywa się w oparciu o przesyłaną parę nagłówków HTTP: `X-Username` (nazwa użytkownika) oraz `X-Api-Token` (unikalny token przypisany do konta). Brak nagłówków lub ich niezgodność skutkuje natychmiastowym przerwaniem żądania i zwróceniem kodu `401 Unauthorized`.

W celu całkowitego wyeliminowania podatności na ataki typu **Mass Assignment** / **Over-Posting**, kontroler API nie komunikuje się bezpośrednio poprzez encje bazodanowe, lecz wykorzystuje dedykowany wzorzec **DTO (Data Transfer Object)** - klasę `WarstwaDTO`, izolując wewnętrzne mechanizmy bazy od klienta REST.

Zestaw udostępnionych endpointów:

Metoda	Adres URL (Endpoint)	Status sukcesu	Opis zachowania
GET	`/api/WarstwaApi`	`200 OK`	Zwraca listę wszystkich warstw sformatowaną do tablicy obiektów JSON.
GET	`/api/WarstwaApi/{id}`	`200 OK`	Zwraca pojedynczą warstwę o wskazanym ID. W przypadku braku zwraca `404 NotFound`.
POST	`/api/WarstwaApi`	`201 Created`	Dodaje nową warstwę na podstawie przekazanego ciała JSON. Klucz główny generowany jest automatycznie przez bazę.
PUT	`/api/WarstwaApi/{id}`	`204 NoContent`	Modyfikuje dane istniejącej warstwy. Wymaga zgodności ID w URL i ciele JSON, chroniąc przed spójnością danych.
DELETE	`/api/WarstwaApi/{id}`	`204 NoContent`	Usuwa warstwę z bazy. Zawiera logikę walidacji: uniemożliwia usunięcie, jeśli do warstwy są przypisane rośliny (`400 BadRequest`).

6. Demonstracja działania - program konsolowy

W celu weryfikacji poprawności działania interfejsu REST API, przygotowany został autonomiczny program konsolowy w języku C# oparty na klasie `HttpWebRequest`. Program symuluje pełny, automatyczny cykl życia zasobu (CRUD) w ramach jednej sesji testowej.

Przebieg demonstracji testowej:

1. **Inicjalizacja:** Uruchomienie żądania GET w celu pobrania i wyświetlenia aktualnego stanu bazy danych. Nagłówki autoryzacyjne dodawane są natywnie do każdego wywołania.
2. **Operacja POST (Tworzenie):** Program serializuje do JSON nowy obiekt warstwy i wysyła go na serwer. Po pomyślnym utworzeniu (210 Created), program parsuje odpowiedź tekstową i dynamicznie wyciąga nowo nadany identyfikator `id`.
3. **Operacja PUT (Aktualizacja):** Wykorzystując pobrane w poprzednim kroku ID, program wysyła zmodyfikowane dane na adres `/api/WarstwaApi/{id}`, weryfikując poprawność aktualizacji stanem 204.

4. **Operacja DELETE (Usuwanie):** Na koniec program wysyła żądanie usunięcia dokładnie tego samego testowego obiektu, przywracając bazę danych do stanu pierwotnego.

> **Wniosek końcowy:** Powyższa architektura w pełni separuje warstwę prezentacji (Widoki Razor MVC) od szyny integracyjnej (REST API), pozwalając na stabilne, bezpieczne i zgodne z najnowszymi standardami inżynierii oprogramowania zarządzanie relacyjną bazą danych.